



Aplicativo de Realidade Aumentada para Educação Ambiental em Maringá

Documentação Técnica - Entrega 2

Protótipo Funcional e Documentação Final

Curso: Análise e Desenvolvimento de Sistemas - 5ª Série

Victor Hideichi Suzuki — RA: 24170759-2

Eduardo Vasconcellos — RA: 24223698-2

Maringá, Junho de 2026

1. Visão Geral do Projeto

O **EcoVision** é um aplicativo mobile desenvolvido com o objetivo de tornar a educação ambiental mais atraente e interativa para jovens e visitantes de parques e museus em Maringá (PR). O aplicativo usa a câmera do celular para identificar elementos naturais — plantas, animais, placas e monumentos — e exibe informações educativas em formato de realidade aumentada.

O projeto está diretamente alinhado ao **ODS 4 — Educação de Qualidade**, da ONU, por promover aprendizado interativo e acessível a todos, sem custo ao usuário.

Funcionalidades Implementadas no Protótipo

#	Funcionalidade	Descrição
1	Scanner AR (simulado)	Identifica elementos naturais pelo código e exibe informações educativas
2	Sistema de Quiz	Perguntas sobre os elementos escaneados com pontuação por acerto
3	Ranking de Usuários	Classificação dos exploradores por pontuação total acumulada
4	Catálogo de Conteúdos	Lista e busca de todos os elementos cadastrados no sistema
5	Login / Cadastro	Autenticação de usuários com email e senha via API REST

2. Programação Avançada — Back-End Spring Boot

2.1 Tecnologias Utilizadas

Tecnologia	Versão	Finalidade
Java	17 (LTS)	Linguagem principal do back-end
Spring Boot	3.2.0	Framework para API REST
Spring Data JPA	3.2.0	Camada de persistência (ORM)
H2 Database	em memória	Banco de dados relacional para desenvolvimento
JUnit 5	5.10	Testes unitários com metodologia TDD
Mockito	5.x	Mock de dependências nos testes

2.2 Endpoints REST Implementados

A API expõe os seguintes endpoints, organizados por recurso:

Método	Endpoint	Descrição
POST	/api/usuarios/cadastro	Cadastra novo usuário
POST	/api/usuarios/login	Realiza login com email e senha
GET	/api/usuarios/{id}	Busca dados de um usuário
GET	/api/usuarios/ranking	Retorna ranking por pontuação (decrecente)
POST	/api/usuarios/{id}/scan	Registra scan realizado pelo usuário
GET	/api/conteudos	Lista todos os conteúdos cadastrados
GET	/api/conteudos/scanner/{codigo}	Busca conteúdo por código do scanner AR
GET	/api/conteudos/tipo/{tipo}	Filtra conteúdos por tipo (PLANTA, ANIMAL, etc.)
GET	/api/quizzes	Lista todos os quizzes disponíveis
GET	/api/quizzes/{id}	Retorna um quiz com todas as suas perguntas
POST	/api/quizzes/{id}/finalizar	Registra pontuação ao finalizar quiz
GET	/api/quizzes/historico/{usuarioId}	Histórico de quizzes do usuário

2.3 Diagrama de Classes — Entidades Principais

O sistema é composto pelas seguintes entidades mapeadas com JPA/Hibernate, demonstrando os conceitos de Orientação a Objetos (encapsulamento, herança de comportamento, associação):

Entidade	Atributos Principais	Relacionamento
Usuario	id, nome, email, senha, pontuacaoTotal, totalScans, dataCadastro	1 para muitos Quizzes, muitos Scans

Conteudo	id, nome, descricao, tipo (enum), categoria, localizacao, pontosPorPergunta	Um para muitos com Quiz
Quiz	id, titulo, pontosPorPergunta	Muitos para um com Conteudo; Um para muitos com Pergunta
Pergunta	id, enunciado, alternativaA/B/C/D, respostaCorreta	Muitos para um com Quiz
Pontuacao	id, pontosObtidos, acertos, totalPerguntas, dataRealizacao	Muitos para um com Usuario e Quiz

2.4 Padrões de Projeto Aplicados

Padrão 1: Singleton

Problema resolvido: garantir que exista apenas uma instância da classe responsável pela configuração do banco de dados, evitando inconsistências e múltiplas conexões desnecessárias.

Implementação: classe *DatabaseConnection* com double-checked locking para segurança em multi-thread.

```
DatabaseConnection instancia = DatabaseConnection.getInstancia();
```

Padrão 2: Factory Method

Problema resolvido: o sistema precisa criar diferentes tipos de conteúdo (PLANTA, ANIMAL, PLACA, MONUMENTO) com configurações específicas para cada tipo, sem espalhar essa lógica pelo código.

Implementação: classe *ConteudoFactory* com métodos estáticos para cada tipo:

```
Conteudo ipe = ConteudoFactory.criarPlanta("Ipe Amarelo", descricao, local, "IPE001");
Conteudo tucano = ConteudoFactory.criarAnimal("Tucano", descricao, local, "TUC001");
```

2.5 Testes Unitários — TDD com JUnit 5

Os testes foram desenvolvidos seguindo a metodologia TDD (Test-Driven Development): primeiro escrevemos os testes para as regras de negócio, depois implementamos o código. Utilizamos Mockito para simular os repositórios nas camadas de serviço.

Classe de Teste	Cenário Testado	Regra de Negócio Validada
UsuarioTests	Cadastro com email único	BusinessException se email duplicado
UsuarioTests	Login com credenciais corretas	Retorna usuario autenticado
UsuarioTests	Login com senha errada	BusinessException senha incorreta
UsuarioTests	Adicionar pontos positivos	Pontuacao aumenta corretamente
UsuarioTests	Adicionar pontos negativos/zero	IllegalArgumentException lançada
UsuarioTests	Busca por ID inexistente	ResourceNotFoundException lançada
UsuarioTests	Ranking ordenado	Lista decrescente por pontuacaoTotal
PerguntaTests	Resposta correta (maiusc./minusc.)	validarResposta() retorna true
PerguntaTests	Resposta incorreta	validarResposta() retorna false
PerguntaTests	Resposta nula/vazia	validarResposta() retorna false
PontuacaoTests	Calcular percentual de acertos	3/5 = 60.0%

ConteudoTests	Busca por codigo valido	Retorna conteudo correto
ConteudoTests	Busca por codigo invalido	ResourceNotFoundException lançada

3. Estruturas, Pesquisa e Ordenação de Dados

3.1 Estruturas de Dados Utilizadas

Estrutura	Onde é usada	Complexidade
Lista (ArrayList)	Histórico de pontuacoes do usuario; lista de perguntas	Busca: $O(n)$
Fila (ArrayDeque/Queue)	Controle da ordem das perguntas no quiz (FIFO)	Enfileirar/Desenfileirar: $O(1)$
Árvore (hierarquia)	Categorização dos conteúdos (Flora/Plantas, Fauna/Animais)	Busca por categoria: $O(\log n)$
Índice (banco de dados)	Busca rápida de conteúdo por codigoReconhecimento	$O(1)$ por índice único
Tabela hash (JPA cache)	Cache de entidades carregadas pelo JPA/Hibernate	Acesso médio: $O(1)$

3.2 Algoritmos de Busca e Ordenação

Ordenação do Ranking — $O(n \log n)$

O ranking de usuários é ordenado pelo banco de dados usando **ORDER BY pontuacaoTotal DESC**. O banco utiliza internamente um algoritmo de ordenação baseado em merge sort ou quick sort, com complexidade $O(n \log n)$, onde n é o número de usuários. Com índice na coluna, o custo pode ser reduzido a $O(1)$ para consultas com paginação.

Busca por Código do Scanner — $O(1)$

A busca de um conteúdo pelo código de reconhecimento (ex: IPE001) utiliza um índice único no banco de dados. Com o índice, a operação de busca tem complexidade constante $O(1)$, independente do número de conteúdos cadastrados.

Busca por Nome (Parcial) — $O(n)$

A busca de conteúdos por nome usa a cláusula SQL LIKE '%texto%', que percorre linearmente todos os registros. Complexidade: $O(n)$. Para grandes volumes, poderia ser otimizado com Full-Text Search.

Fila de Perguntas — $O(1)$ por operação

A estrutura Queue (ArrayDeque) é usada para controlar a ordem de apresentação das perguntas. As operações de enfileirar (offer) e desenfileirar (poll) têm complexidade $O(1)$, garantindo que as perguntas sejam apresentadas na ordem correta sem custo computacional adicional.

4. Programação para Dispositivos Móveis — Flutter

4.1 Justificativa da Plataforma

Foi escolhida a plataforma **cross-platform com Flutter (Dart)**, conforme definido na Entrega 1. A escolha se mantém justificada pelos seguintes motivos:

- Um único código-fonte funciona tanto em Android quanto em iOS
- Flutter oferece suporte nativo à câmera via pacote *camera*
- Desempenho próximo ao nativo graças à compilação para código de máquina
- Reduz o tempo e custo de desenvolvimento para equipes pequenas

4.2 Telas Implementadas no Protótipo

Tela	Arquivo	Funcionalidade
Login	login_screen.dart	Autenticação ou acesso como visitante
Home	home_screen.dart	Painel do usuário com estatísticas e menu de navegação
Scanner AR	scanner_screen.dart	Simulação do escaneamento e exibição de informações
Lista de Quizzes	quiz_screen.dart	Listagem dos quizzes disponíveis
Jogo de Quiz	quiz_screen.dart	Interface de perguntas e respostas com pontuação
Ranking	ranking_screen.dart	Podium e lista completa de usuários por pontuação
Catálogo	catalogo_screen.dart	Lista de elementos com busca por nome/categoria

4.3 Comunicação com o Back-End

O arquivo *api_service.dart* centraliza todas as chamadas HTTP à API REST. Utiliza o pacote **http** do Flutter para requisições assíncronas (*async/await*). O endereço base da API é configurável: para emulador Android usa-se *10.0.2.2:8080* (redirecionamento do localhost do computador).

5. Interface Humano-Computador (IHC)

5.1 Protótipo de Alta Fidelidade — Descrição das Telas

O aplicativo foi desenvolvido com Material Design 3 (via Flutter), com paleta de cores baseada no verde — cor diretamente associada à natureza e sustentabilidade.

Tela	Elementos Principais	Decisão de Design
Login	Logo EcoVision, campos de email/senha	Acesso rápido e seguro; cadastro obrigatório — reduz barreira de entrada
Home	Card de boas-vindas com pontos, grid 2x2 de funcionalidades	Exibição clara de pontos; facilidade de descoberta das funcionalidades
Scanner	Moldura animada com cantos verdes, lista de itens escaneados	Feedback visual imediato; identificação clara do objeto; informações claras
Quiz	Barra de progresso, card da pergunta, alternativas	Progresso visualizado; feedback mantém engajamento
Ranking	Podium animado para top 3, lista com detalhes	Estímulo à competição; usuário vê sua posição facilmente

5.2 Avaliação Heurística de Usabilidade (Nielsen)

A equipe realizou uma avaliação heurística baseada nas 10 heurísticas de Jakob Nielsen:

Heurística	Aplicação no EcoVision	Avaliação
1. Visibilidade do status	Indicador de carregamento, barra de progresso no Quiz	Atendida
2. Correspondência com o mundo real	Ícones intuitivos (câmera, quiz, troféu, livro)	Atendida
3. Controle do usuário	Botão voltar em todas as telas; opção de visitante	Atendida
4. Consistência e padrões	Cores, tipografia e espaçamento uniformes em todas as telas	Atendida
5. Prevenção de erros	Validação de campos antes do envio; mensagens de erro claras	Atendida
6. Reconhecimento vs memória	Menús sempre visíveis na barra inferior; labels de ações	Atendida
7. Flexibilidade e eficiência	Acesso rápido via botões na Home; busca no catálogo	Atendida
8. Design minimalista	Sem informações desnecessárias; foco no conteúdo principal	Atendida
9. Recuperação de erros	Mensagem amigável em caso de falha na API; modo offline parcial	Parcial
10. Ajuda e documentação	Dica do dia na Home; tooltips nos botões de ação	Parcial

5.3 Fluxo de Interação do Usuário

O fluxo principal de uso do EcoVision segue o caminho:

Login → **Home** → **Scanner AR** (seleciona elemento) → **Visualiza informações** → (opcional) **Quiz** → **Resultado com pontuação** → **Ranking**

Cada scan realizado incrementa o contador de scans do usuário no back-end. Ao completar um quiz, os pontos são somados automaticamente ao perfil do usuário, atualizando sua posição no ranking em tempo real.

6. Manual de Instalação e Execução

6.1 Requisitos do Sistema

Componente	Versão Mínima	Observação
Java (JDK)	17 ou superior	Necessário para rodar o Spring Boot
Apache Maven	3.8+	Gerenciador de dependências do back-end
Flutter SDK	3.0+	Necessário para rodar o app mobile
Android SDK	API 21+ (Android 5.0)	Para rodar no emulador ou dispositivo
IDE recomendada	IntelliJ IDEA / VS Code	Com plugins Java e Flutter instalados

6.2 Executando o Back-End

Abra o terminal e execute os comandos na pasta do projeto:

```
# 1. Entre na pasta do back-end cd ecovision-backend # 2. Execute o projeto com Maven
mvn spring-boot:run # O servidor iniciará em: http://localhost:8080 # Console H2 (banco
de dados): http://localhost:8080/h2-console # URL: jdbc:h2:mem:ecovisiondb | Usuario:
sa | Senha: (vazio)
```

6.3 Executando os Testes

```
# Executa todos os testes unitários JUnit 5 mvn test # Resultado esperado: # Tests run:
14, Failures: 0, Errors: 0, Skipped: 0
```

6.4 Executando o App Flutter

```
# 1. Entre na pasta do app cd ecovision-flutter # 2. Instale as dependências flutter pub
get # 3. Inicie um emulador Android ou conecte um dispositivo físico # 4. Execute o
aplicativo flutter run # Para gerar o APK de instalação: flutter build apk --debug
```

Importante: O back-end deve estar rodando antes de iniciar o app. No emulador Android, o endereço `10.0.2.2:8080` redireciona automaticamente para o localhost do computador onde o Spring Boot está executando.

7. Relatório de Imersão Profissional — Fábrica de Software

7.1 Sprint Retrospectivas

Sprint	Período	O que funcionou	O que melhorar
Sprint 1 (Planejamento)	Abril	Definição clara do escopo e das tecnologias.	Podemos começar a trabalhar mais cedo nos wireframes desde o início.
Sprint 2 (Back-End)	Maio (semanas 1-2)	Spring Boot facilitou a criação dos endpoints.	Podemos definir regras de negócio mais cedo.
Sprint 3 (Mobile)	Maio (semanas 3-4)	Flutter permitiu criar as telas rapidamente.	Podemos fazer a integração de estado antes. Poderia usar Riverpod ou BLoC.
Sprint 4 (Testes e Docs)	Junho	Todos os testes unitários passando; documentação completa.	Podemos fazer testes de integração e teste de usuário mais cedo.

7.2 Canvas de Proposta de Valor Atualizado

Dimensão	Conteúdo
Problema	Educação ambiental apresentada de forma pouco interativa em parques e museus de Maringá.
Solução	App com scanner AR, quizzes gamificados e ranking para engajar estudantes e visitantes.
Público-alvo	Estudantes de 10 a 25 anos; visitantes de parques e museus; educadores ambientais.
Proposta de Valor	Aprender sobre a natureza de Maringá de forma divertida, gratuita e interativa.
Canais	Google Play (APK), distribuição em escolas parceiras, QR codes nos parques.
Receita	Gratuito ao usuário; possível parceria com Prefeitura de Maringá e patrocinadores.
Parceiros	Prefeitura de Maringá, Parque do Ingá, escolas públicas e privadas da região.
Impacto ODS	ODS 4 (Educação de Qualidade) — promove aprendizado acessível e engajante.

7.3 Product Backlog Simplificado — Próximos Passos

Funcionalidades planejadas para versões futuras do EcoVision:

Prioridade	Funcionalidade	Valor ao Usuário
Alta	Reconhecimento visual real com ML (TensorFlow Lite)	Feedback automático sem seleção manual
Alta	Conteúdo offline (download de pacotes)	Uso sem internet nos parques
Alta	Notificações de conquistas e novos conteúdos	Maior retenção e engajamento
Média	Modo multiplayer de quiz (desafiar amigos)	Gamificação social
Média	Mapa interativo dos elementos no parque	Orientação física no espaço
Média	Painel administrativo para gestores de parques	Cadastro de novos conteúdos sem código
Baixa	Versão iOS nativa (mesmo código Flutter)	Ampliar base de usuários

Baixa	Integração com sistemas educacionais das escolas	Classe formal em sala de aula
-------	--	-------------------------------

7.4 Reflexão sobre Contribuição aos ODS

O EcoVision contribui diretamente para o **ODS 4 — Educação de Qualidade** ao oferecer uma ferramenta educacional gratuita, acessível e interativa para todos os visitantes dos parques e museus de Maringá. A gamificação (pontos, ranking, quizzes) aumenta o engajamento, especialmente entre jovens, que costumam se interessar pouco por métodos tradicionais de educação ambiental.

Indiretamente, o projeto também contribui para o **ODS 11 — Cidades e Comunidades Sustentáveis**, ao modernizar os espaços públicos de Maringá com tecnologia acessível, valorizando o patrimônio natural e cultural da cidade. O aumento da consciência ambiental também impacta positivamente o **ODS 15 — Vida Terrestre**, ao educar a população sobre a importância da biodiversidade local.

8. Conclusão

O protótipo funcional do EcoVision demonstra que é possível desenvolver uma solução tecnológica educacional relevante utilizando ferramentas amplamente disponíveis e tecnologias modernas do mercado. O back-end em Java Spring Boot fornece uma API REST estável e bem testada, enquanto o app Flutter oferece uma experiência visual agradável e consistente em dispositivos Android.

As cinco disciplinas do curso foram integradas de forma coerente: a lógica de negócio em Java com Spring Boot (Programação Avançada), a organização dos dados com estruturas como filas e árvores de categorias (Estruturas de Dados), a interface intuitiva e heurística (IHC), o desenvolvimento mobile cross-platform (Programação para Dispositivos Móveis) e o planejamento com Scrum e Canvas de negócios (Imersão Profissional).